



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №9
на тему «РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-
SERVER, PEER-TO-PEER, SERVICE-ORIENTED
ARCHITECTURE»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мягкий Михайло Юрійович

Зміст

Тема	3
Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми	8
Реалізація архітектури за обраною темою	11
1. EditService.....	11
2. EditController	11
3. FileService.....	13
4. FileController	14
5. FlyweightService	16
6. FlyweightController.....	16
7. MacroService.....	17
8. MacroController	18
9. NavigationService	19
10. NavigationController	19
11. SyntaxHighlightService	20
12. SyntaxHighlightController.....	21
13. DefaultController	22
Посилання на код на GitHub	22
Діаграма класів архітектури «SOA»	23
Переваги застосування архітектури «SOA»	24

РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE

Мета: ознайомитися з різними взаємодіями додатків: client-server, peer-to-peer, service-oriented architecture та набутти практичних навичок їх застосування. Реалізувати частину функціоналу робочої програми у вигляді класів і їх взаємодій для досягнення конкретних функціональних можливостей.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Хід роботи:

Короткі теоретичні відомості:

Архітектура додатків визначає спосіб організації компонентів програмного забезпечення та їх взаємодії для досягнення функціональних і нефункціональних вимог системи. Вона задає структуру, правила й стандарти для побудови додатків, забезпечуючи масштабованість, ефективність і легкість супроводу.

Основні принципи архітектури додатків:

1. **Розподіл обов'язків:** різні компоненти виконують конкретні функції, що підвищує модульність і зручність супроводу.
2. **Взаємодія через стандартизовані інтерфейси:** забезпечує сумісність між компонентами незалежно від платформи чи технології.
3. **Масштабованість:** архітектура має підтримувати адаптацію до зростання навантаження.
4. **Незалежність компонентів:** мінімізація залежностей між частинами системи для спрощення оновлень і розширення.

Типи архітектур відрізняються способами взаємодії компонентів:

- У **централізованих** архітектурах є чітко визначений центр управління.

- У **розподілених** архітектурах компоненти взаємодіють через мережу, що дає більшу гнучкість і стійкість до відмов.
- **Гібридні** архітектури комбінують підходи для задоволення специфічних потреб.

Вибір архітектури залежить від вимог проєкту, таких як масштабованість, надійність, продуктивність і простота супроводу. Основні моделі, такі як клієнт-сервер, peer-to-peer і сервіс-орієнтована архітектура, реалізують ці принципи з урахуванням різних сценаріїв використання.

Клієнт-серверні додатки

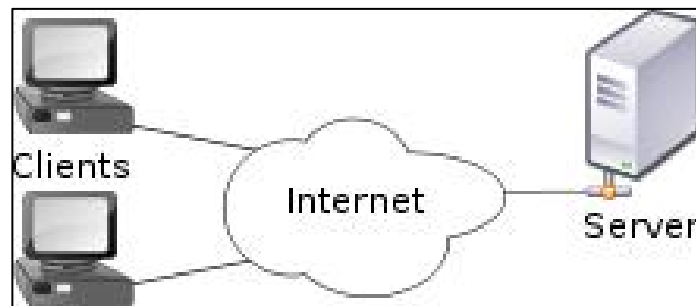


Рисунок 1. Архітектура «Client-server»

Клієнт-серверні додатки – це розподілені системи, що складаються з клієнтів (для взаємодії з користувачем) та серверів (для обробки й зберігання даних). Розрізняють **тонкі** та **товсті** клієнти.

Тонкий клієнт передає більшість обчислень на сервер, залишаючи на собі лише візуальне відображення даних. Це зменшує вимоги до клієнтських пристроїв, але збільшує навантаження на сервер. Така модель актуальна для захищених сценаріїв (захист даних від зламу) або для уникнення конфліктів при множинному доступі до даних.

Товстий клієнт виконує більшість обчислень локально, розвантажуючи сервер, який виступає лише як точка доступу до ресурсів (наприклад, бази даних).

Для взаємодії використовується модель «підписка/видача», де клієнти підписуються на події, а сервер їх сповіщає. Це зручно для оновлення даних, але створює навантаження на сервер і канали зв'язку.

Зазвичай клієнт-серверна структура організовується на 3 рівні:

- **Клієнтська частина:** відображення даних, обробка дій користувача, зв'язок із сервером.

- **Загальна частина (middleware):** спільні класи й компоненти.
- **Серверна частина:** бізнес-логіка, зберігання та обмін даними.

Додатки типу «peer-to-peer»

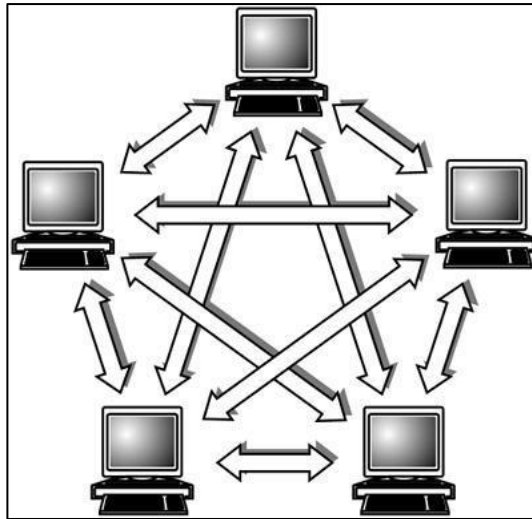


Рисунок 2. Архітектура «Peer-to-peer»

Модель взаємодії додатків типу реєр - to - реєр має на увазі рівноправ'я клієнтських програм і відсутність серверної програми. Усі клієнтські програми контактують між собою для виконання спільних цілей. У таких мережах найчастіше виникають наступні проблеми: синхронізація даних, пошук клієнтських застосунків.

Для пошуку клієнтських застосунків може використовуватися одна загальна адреса, що містить набір адрес підключених клієнтів. Це в деякому роді нагадує сервер. Проте сервер є лише центром сходження безлічі клієнтів, а адреси можуть зберігатися в самих клієнтських застосунках. Це називається структуровані однорангові мережі.

Для вирішення проблеми синхронізації даних використовуються традиційні підходи: можливий варіант договору про набір для синхронізації (negotiation), або визначення по деякому алгоритму порівняння даних (hash- алгоритми і інші). Як правило, реєр - to - реєр застосування є звичайними застосунками з продуманою інфраструктурою обміну повідомленнями між клієнтами. Для таких випадків як правило розробляються спеціальні формати і протоколи обміну для структуризації діалогу між клієнтами.

Сервіс-орієнтована архітектура

Сервіс-орієнтована архітектура (SOA, service-oriented architecture) — це підхід до розробки програмного забезпечення, який передбачає використання розподілених, слабо пов'язаних компонентів із стандартизованими інтерфейсами для взаємодії через стандартизовані протоколи. SOA дозволяє створювати масштабовані, керовані системи, незалежні від платформи та інструментів розробки. Програмні комплекси, побудовані за SOA, зазвичай реалізуються як набір веб-служб, що використовують протоколи SOAP або REST. Інтерфейси компонентів інкапсулюють деталі реалізації, що забезпечує можливість багаторазового використання та легкість інтеграції.

SaaS (Software as a Service) — це бізнес-модель, у якій постачальник розробляє веб-додаток, самостійно ним керує та надає клієнтам доступ через Інтернет.

Основні риси SaaS:

- Відсутність витрат на встановлення, оновлення й підтримку ПЗ та обладнання.
- Один додаток може обслуговувати кілька клієнтів.
- Оплата здійснюється як абонентська плата або залежить від обсягу використання.
- Оновлення та техпідтримка включені у вартість.
- Оновлення відбуваються автоматично та прозоро для клієнтів.

SaaS дозволяє клієнтам зменшити витрати, адже замість купівлі програмного забезпечення й апаратного забезпечення для його розгортання, вони орендують доступ до ПЗ. Це робить модель привабливою для тих, хто потребує гнучкості. При необхідності можна тимчасово припинити використання програми та оплати. Для розробників SaaS знижує витрати на підтримку та впровадження ПЗ, а також допомагає боротися з неліцензійним використанням, оскільки доступ до додатку здійснюється тільки через Інтернет.

Реалізація SaaS базується на SOA та зазвичай є stateless (без стану). Для авторизації користувачів використовуються токени, які видаються після успішного входу. Зберігання даних і обробка запитів виконуються в хмарі, що забезпечує гнучкість і масштабованість систем.

Мікро-сервісна архітектура.

Архітектура мікрослужб — це підхід до розробки додатків, де система складається з невеликих автономних служб. Кожна служба виконується у власному процесі та взаємодіє з іншими через стандартизовані протоколи, такі як HTTP/HTTPS, WebSockets чи AMQP. Служби створюються для виконання

конкретних бізнес-завдань у межах обмеженого контексту, розробляються незалежно та можуть бути розгорнуті автономно.

Мікрослужби забезпечують гнучкість і масштабованість у довгостроковій перспективі. Вони дозволяють розробляти, оновлювати й масштабувати систему частинами, що спрощує підтримку складних програм. Завдяки автономності, кожна мікрослужба може використовувати власну технологію, базу даних або мову програмування.

Переваги:

- **Масштабованість:** кожна служба масштабується незалежно.
- **Гнучкість:** використання різних технологій для різних завдань.
- **Супровідність:** код служби зрозумілий і легкий для змін.
- **Стійкість:** збої в одній службі не впливають на роботу інших.
- **Незалежна розробка:** команди працюють над різними службами паралельно.

Виклики:

- **Управління складністю:** потрібні інструменти для моніторингу, оркестрації та забезпечення безпеки.
- **Взаємодія між службами:** важливо забезпечити надійність і швидкість передачі даних.
- **Розподілена природа:** ускладнюється тестування та підтримка узгодженості даних.

Архітектура мікрослужб добре підходить для масштабних і динамічних проєктів, де потрібна швидка адаптація до змін. Вона часто застосовується у хмарних середовищах із використанням таких інструментів, як Docker, Kubernetes, RabbitMQ та API-шлюзи, що допомагають автоматизувати розгортання й управління.

Реалізація не менше 3-х класів відповідно до обраної теми:

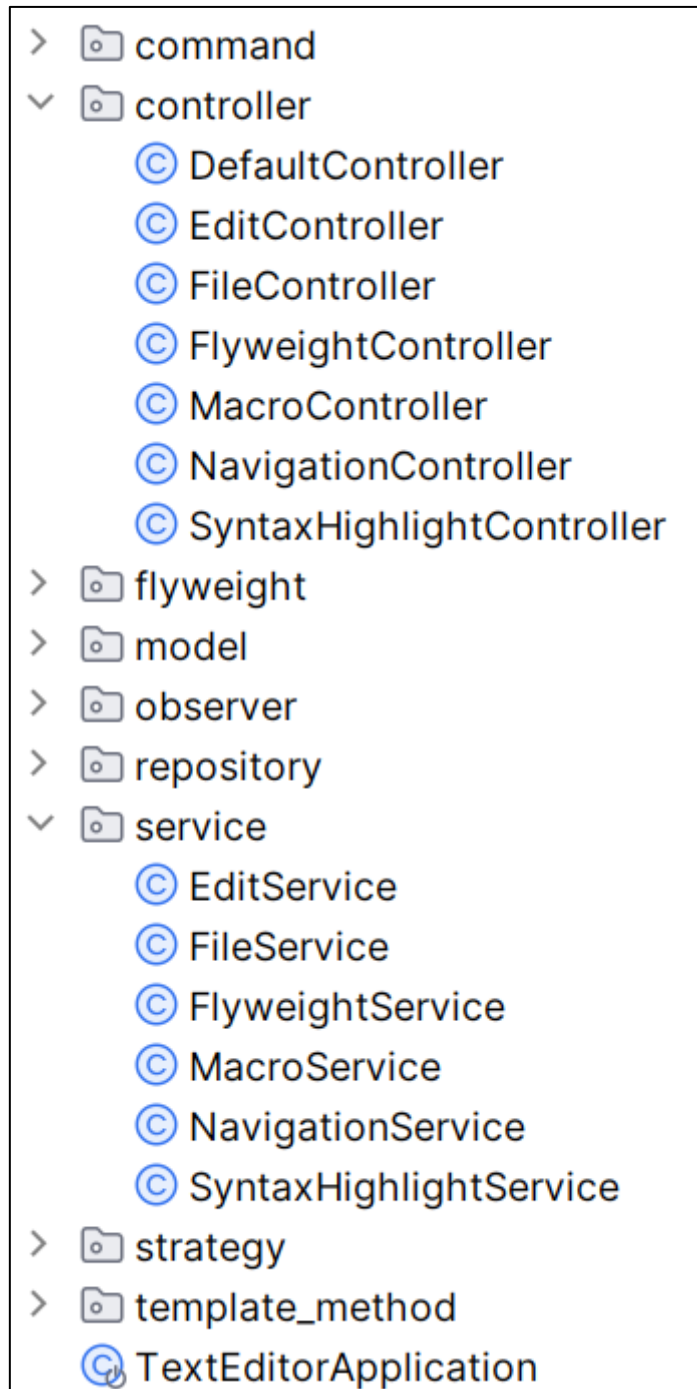


Рисунок 3. Структура проекту з архітектурою «SOA».

У ході лабораторної роботи була реалізована система текстового редактора з використанням архітектури Service-Oriented Architecture (SOA), яка забезпечує модульну взаємодію між компонентами програми через стандартизовані інтерфейси. Основним завданням було впровадження SOA для побудови гнучкої та масштабованої системи, що дозволяє незалежно розгортати та модернізувати сервіси. Структура проєкту наведена на Рисунок 1. У результаті виконання роботи були реалізовані наступні кроки::

1. Розроблено структуру проєкту:

- Проєкт поділено на логічні пакети: controller, service, model, repository, strategy та інші, що забезпечує зручність масштабування та підтримки коду.
- Для кожного сервісу створено власний REST API, який відповідає за взаємодію із зовнішніми компонентами.

2. Реалізовано архітектуру SOA:

- EditService - відповідає за редагування тексту.
- FileService - забезпечує роботу з файлами текстового редактора.
- FlyweightService - оптимізує збереження пам'яті для шрифтів.
- MacroService - відповідає за обробку макросів у текстовому редакторі.
- NavigationService - відповідає за роботу із закладками та навігацію.
- SyntaxHighlightService - забезпечує підсвічування синтаксису у текстовому редакторі.

3. Реалізовано REST API для взаємодії:

- Для кожного сервісу створено окремий контролер:
 - DefaultController
 - **GET /:** Вітає користувача та перелічує доступні ендпойнти системи (/file, /editor, /syntax, /navigation).
 - EditController
 - **POST /editor/save:** Збереження файлу, приймає назву файлу та його вміст.
 - **POST /editor/insert:** Додавання нового тексту до наявного, приймає поточний текст та новий фрагмент.
 - FileController
 - **GET /file/open:** Завантаження файлу за його назвою. Якщо файл не знайдено, створюється новий із дефолтним вмістом.
 - **POST /file/save:** Збереження файлу, приймає назву файлу та його вміст.
 - **PUT /file/edit:** Оновлення вмісту файлу, приймає назву файлу та новий вміст.
 - FlyweightController
 - **POST /flyweight/display:** Відображення тексту з параметрами шрифту (назва, розмір).
 - **GET /flyweight/count:** Повертає кількість створених об'єктів Flyweight.
 - MacroController
 - **POST /macros/process/formatting:** Обробка макросу для форматування тексту.

- **POST /macros/process/snippet**: Обробка макросу для сніпетів тексту.
- **NavigationController**
 - **POST /navigation/bookmark**: Додавання закладки до файлу за його назвою, вказаним рядком та коментарем.
 - **GET /navigation/bookmarks**: Отримання списку закладок для заданого файлу.
- **SyntaxHighlightController**
 - **POST /syntax/highlight**: Підсвічування синтаксису для тексту.

4. Перевірено функціонал системи:

- Тестування API через Postman підтвердило коректність взаємодії між сервісами.
- Проведено перевірку масштабованості: сервіси працюють незалежно, забезпечуючи високу модульність та гнучкість.
- Перевірено роботу з великим обсягом текстових файлів, різними мовами програмування та користувацькими запитами.

Цей підхід забезпечує високий рівень гнучкості, повторне використання та незалежність компонентів. Завдяки впровадженню SOA текстовий редактор став більш масштабованим і придатним для подальшого розвитку.

Архітектура дозволяє легко додавати нові сервіси та функції, не впливаючи на існуючі модулі.

Реалізація архітектури за обраною темою:

1. EditService

```
1 package com.example.text_editor.service;
2
3 import com.example.text_editor.observer.FileSubject;
4 import org.springframework.stereotype.Service;
5
6 6 usages
7 @Service
8 public class EditService {
9     2 usages
10     private final FileSubject fileSubject;
11
12     1 usage
13     public EditService(FileSubject fileSubject) { this.fileSubject = fileSubject; }
14
15     public String insertText(String currentContent, String newText) {
16         String updatedContent = currentContent + "\n" + newText;
17         fileSubject.setContent(updatedContent);
18         return updatedContent;
19     }
20 }
```

Рисунок 4. Клас EditService.

Цей клас, показаний на Рисунку 4, реалізує функціонал для редагування тексту. Основний метод `insertText` дозволяє додавати новий текст до поточного вмісту. Він також сповіщає підписників про зміни у вмісті через об'єкт `FileSubject`.

2. EditController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.command.*;
4 import com.example.text_editor.service.EditService;
5 import com.example.text_editor.service.FileService;
6 import org.slf4j.Logger;
7 import org.slf4j.LoggerFactory;
8 import org.springframework.web.bind.annotation.*;
9
10 @RestController
11 @RequestMapping("/editor")
12 public class EditController {
13
14     8 usages
15     private static final Logger logger = LoggerFactory.getLogger(EditController.class);
16
17     3 usages
18     private final CommandInvoker commandInvoker;
19     2 usages
20     private final FileService fileService;
21     2 usages
22     private final EditService editService;
23
24     public EditController(CommandInvoker commandInvoker, FileService fileService, EditService editService) {
25         this.commandInvoker = commandInvoker;
26         this.fileService = fileService;
27         this.editService = editService;
28     }
29 }
```

Рисунок 5.1. Клас EditController.

```

26      @PostMapping("/save")
27      public String saveFile(@RequestParam String fileName, @RequestParam String content) {
28          if (fileName == null || fileName.isEmpty()) {
29              logger.error("File name is missing.");
30              return "Error: File name is required.";
31          }
32          if (content == null) {
33              logger.error("Content is missing.");
34              return "Error: Content is required.";
35          }
36
37          try {
38              Command saveCommand = new SaveCommand(fileService, fileName, content);
39              commandInvoker.executeCommand(saveCommand);
40              logger.info("File '{}' saved successfully.", fileName);
41              return "File saved successfully.";
42          } catch (Exception e) {
43              logger.error("Error while saving file '{}': {}", fileName, e.getMessage());
44              return "Error: Could not save file.";
45          }
46      }

```

Рисунок 5.2. Клас EditController.

```

48      @PostMapping("/insert")
49      public String insertText(@RequestParam String currentContent, @RequestParam String newText) {
50          if (currentContent == null) {
51              logger.error("Current content is missing.");
52              return "Error: Current content is required.";
53          }
54          if (newText == null || newText.isEmpty()) {
55              logger.error("New text is missing.");
56              return "Error: New text is required.";
57          }
58
59          try {
60              Command insertCommand = new InsertTextCommand(editService, currentContent, newText);
61              commandInvoker.executeCommand(insertCommand);
62              logger.info("Text inserted successfully.");
63              return "Text inserted successfully.";
64          } catch (Exception e) {
65              logger.error("Error while inserting text: {}", e.getMessage());
66              return "Error: Could not insert text.";
67          }
68      }
69  }

```

Рисунок 5.3. Клас EditController.

Цей контролер, показаний на Рисунках 5.1, 5.2 та 5.3, надає API для роботи з функціоналом редагування тексту. Метод `saveFile` дозволяє зберігати файл через HTTP-запити, а метод `insertText` — додавати новий текст до наявного. У разі некоректних параметрів обробляє помилки та повертає відповідні повідомлення.

3. FileService

```
1 package com.example.text_editor.service;
2
3 import com.example.text_editor.observer.FileSubject;
4 import com.example.text_editor.model.TextFile;
5 import com.example.text_editor.repository.TextFileRepository;
6 import org.springframework.stereotype.Service;
7
8 9 usages
9 @Service
10 public class FileService {
11     7 usages
12     private final TextFileRepository textFileRepository;
13     4 usages
14     private final FileSubject fileSubject;
15
16     public FileService(TextFileRepository textFileRepository, FileSubject fileSubject) {
17         this.textFileRepository = textFileRepository;
18         this.fileSubject = fileSubject;
19     }
20
21     1 usage
22     public TextFile openFile(String fileName) {
23         TextFile file = textFileRepository.findByName(fileName);
24         if (file == null) {
25             file = new TextFile(fileName, content: "Empty content");
26             textFileRepository.save(file);
27         }
28         fileSubject.setContent(file.getContent());
29         return file;
30     }
31 }
```

Рисунок 6.1. Клас FileService.

```
1 usage
28 public TextFile saveFile(String fileName, String content) {
29     TextFile file = textFileRepository.findByName(fileName);
30     if (file == null) {
31         file = new TextFile(fileName, content);
32     } else {
33         file.setContent(content);
34     }
35     TextFile savedFile = textFileRepository.save(file);
36     fileSubject.setContent(content);
37     return savedFile;
38 }
39
40 1 usage
41 public TextFile editFile(String fileName, String newContent) {
42     TextFile file = textFileRepository.findByName(fileName);
43     if (file != null) {
44         file.setContent(newContent);
45         textFileRepository.save(file);
46         fileSubject.setContent(newContent);
47         return file;
48     }
49     throw new RuntimeException("File not found");
50 }
```

Рисунок 6.1. Клас FileService.

Цей клас, показаний на Рисунках 6.1 та 6.2, відповідає за управління текстовими файлами. Метод `openFile` завантажує файл із репозиторію або створює новий, якщо файл відсутній. Метод `saveFile` дозволяє зберігати або оновлювати файл із вказаним вмістом. Метод `editFile` дозволяє оновлювати вміст існуючого файлу. Усі зміни у вмісті файлів сповіщаються через `FileSubject`.

4. FileController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.command.CommandInvoker;
4 import com.example.text_editor.command.SaveCommand;
5 import com.example.text_editor.model.TextFile;
6 import com.example.text_editor.service.FileService;
7 import org.slf4j.Logger;
8 import org.slf4j.LoggerFactory;
9 import org.springframework.web.bind.annotation.*;
10
11 @RestController
12 @RequestMapping("/file")
13 public class FileController {
14
15     8 usages
16     private static final Logger logger = LoggerFactory.getLogger(FileController.class);
17
18     4 usages
19     private final FileService fileService;
20
21     2 usages
22     private final CommandInvoker commandInvoker;
23
24     public FileController(FileService fileService, CommandInvoker commandInvoker) {
25         this.fileService = fileService;
26         this.commandInvoker = commandInvoker;
27     }
28 }
```

Рисунок 7.1. Клас `FileController`.

```

25     @GetMapping("/{open}")
26     public TextFile openFile(@RequestParam String fileName) {
27         if (fileName == null || fileName.isEmpty()) {
28             logger.error("File name is missing.");
29             throw new IllegalArgumentException("File name cannot be empty.");
30         }
31
32         TextFile file = fileService.openFile(fileName);
33         logger.info("File '{}' opened successfully.", fileName);
34         return file;
35     }
36
37     @PostMapping("/{save}")
38     public String saveFile(@RequestParam String fileName, @RequestParam String content) {
39         if (fileName == null || fileName.isEmpty()) {
40             logger.error("File name is missing.");
41             throw new IllegalArgumentException("File name cannot be empty.");
42         }
43         if (content == null) {
44             logger.error("File content is missing.");
45             throw new IllegalArgumentException("File content cannot be null.");
46         }
47
48         SaveCommand saveCommand = new SaveCommand(fileService, fileName, content);
49         commandInvoker.executeCommand(saveCommand);
50         logger.info("File '{}' saved successfully.", fileName);
51         return "File saved successfully.";
52     }

```

Рисунок 7.2. Клас FileController.

```

54     @PutMapping("/{edit}")
55     public String editFile(@RequestParam String fileName, @RequestParam String newContent) {
56         if (fileName == null || fileName.isEmpty()) {
57             logger.error("File name is missing.");
58             throw new IllegalArgumentException("File name cannot be empty.");
59         }
60         if (newContent == null) {
61             logger.error("New content is missing.");
62             throw new IllegalArgumentException("New content cannot be null.");
63         }
64
65         TextFile file = fileService.editFile(fileName, newContent);
66         logger.info("File '{}' edited successfully. New content: {}", fileName, newContent);
67         return "File edited successfully: " + file.getName();
68     }
69 }

```

Рисунок 7.3. Клас FileController.

Цей контролер, показаний на Рисунках 7.1, 7.2 та 7.3, надає API для управління файлами. Метод openFile завантажує файл за його назвою, saveFile зберігає файл із вказаним вмістом, а editFile оновлює вміст файлу.

5. FlyweightService

```
1 package com.example.text_editor.service;
2
3 import com.example.text_editor.flyweight.FontFlyweight;
4 import com.example.text_editor.flyweight.FontFlyweightFactory;
5 import org.springframework.stereotype.Service;
6
7 @Service
8 public class FlyweightService {
9     @
10     public void displayText(String text, String fontName, int fontSize) {
11         FontFlyweight font = FontFlyweightFactory.getFont(fontName, fontSize);
12         for (char character : text.toCharArray()) {
13             System.out.println("Character: " + character + ", Font: " + font.getFontName() + ", Size: " + font.getFontSize());
14         }
15     }
16     public int getFlyweightCount() { return FontFlyweightFactory.getFlyweightCount(); }
17 }
18
19 }
```

Рисунок 8. Клас FlyweightService.

Цей клас, показаний на Рисунок 8, реалізує логіку роботи зі шрифтами, використовуючи патерн "Flyweight". Метод `displayText` демонструє текст із використанням об'єктів `Flyweight`, зберігаючи пам'ять. Метод `getFlyweightCount` повертає кількість створених об'єктів `Flyweight`.

6. FlyweightController

```
1 package com.example.text_editor.controller;
2
3 import org.springframework.http.ResponseEntity;
4 import org.springframework.web.bind.annotation.*;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import com.example.text_editor.service.FlyweightService;
7
8 @RestController
9 @RequestMapping("/flyweight")
10 public class FlyweightController {
11
12     3 usages
13     private final FlyweightService flyweightService;
14
15     @Autowired
16     public FlyweightController(FlyweightService flyweightService) { this.flyweightService = flyweightService; }
17
18     @PostMapping("/display")
19     public ResponseEntity<String> displayText(
20         @RequestParam(required = false) String text,
21         @RequestParam(required = false) String fontName,
22         @RequestParam(required = false) Integer fontSize) {
23         if (text == null || text.isEmpty()) {
24             return ResponseEntity.badRequest().body("Error: 'text' parameter is missing or empty.");
25         }
26         if (fontName == null || fontName.isEmpty()) {
27             return ResponseEntity.badRequest().body("Error: 'fontName' parameter is missing or empty.");
28         }
29         if (fontSize == null || fontSize <= 0) {
30             return ResponseEntity.badRequest().body("Error: 'fontSize' parameter is missing or invalid.");
31         }
32
33         flyweightService.displayText(text, fontName, fontSize);
34         return ResponseEntity.ok("Text displayed successfully!");
35     }
36 }
```

Рисунок 9.1. Клас FlyweightController.


```

39     @GetMapping("/count")
40     public String getFlyweightCount() {
41         return "Number of flyweights created: " + flyweightService.getFlyweightCount();
42     }
43 }

```

Рисунок 9.2. Клас FlyweightController.

Цей контролер, показаний на Рисунках 9.1 та 9.2, надає API для роботи з Flyweight-об'єктами. Метод displayText приймає текст і параметри шрифту для відображення через Flyweight, а метод getFlyweightCount повертає кількість створених об'єктів.

7. MacroService

```

1  package com.example.text_editor.service;
2
3  import com.example.text_editor.template_method.FormattingMacroProcessor;
4  import com.example.text_editor.template_method.MacroProcessor;
5  import com.example.text_editor.template_method.SnippetMacroProcessor;
6  import org.springframework.stereotype.Service;
7
8  3 usages
9  @Service
10 public class MacroService {
11
12     1 usage
13     public void processFormattingMacro(String macroText) {
14         MacroProcessor processor = new FormattingMacroProcessor();
15         processor.processMacro(macroText);
16     }
17
18     1 usage
19     public void processSnippetMacro(String macroText) {
20         MacroProcessor processor = new SnippetMacroProcessor();
21         processor.processMacro(macroText);
22     }
23 }

```

Рисунок 10. Клас MacroService.

Цей клас, показаний на Рисунку 10, реалізує обробку макросів у текстовому редакторі. Метод processFormattingMacro дозволяє обробляти макроси для форматування тексту, а метод processSnippetMacro — для роботи зі сніпетами коду. Логіка реалізована за допомогою патерну "Template Method".

8. MacroController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.service.MacroService;
4 import org.springframework.web.bind.annotation.*;
5
6 import java.util.Map;
7
8 @RestController
9 @RequestMapping("/macros")
10 public class MacroController {
11
12     3 usages
13     private final MacroService macroService;
14
15     public MacroController(MacroService macroService) { this.macroService = macroService; }
16
17     @PostMapping("/process/formatting")
18     public void processFormattingMacro(@RequestBody Map<String, String> request) {
19         String macroText = request.get("macroType");
20         if (macroText != null) {
21             macroService.processFormattingMacro(macroText);
22         } else {
23             System.out.println("Invalid request: macroType is missing");
24         }
25     }
26
27     @PostMapping("/process/snippet")
28     public void processSnippetMacro(@RequestBody Map<String, String> request) {
29         String macroText = request.get("macroType");
30         if (macroText != null) {
31             macroService.processSnippetMacro(macroText);
32         } else {
33             System.out.println("Invalid request: macroType is missing");
34         }
35     }
36 }
37 }
```

Рисунок 11. Клас MacroController.

Цей контролер, показаний на Рисунку 11, забезпечує API для обробки макросів. Метод processFormattingMacro дозволяє обробляти макроси для форматування, а метод processSnippetMacro — для роботи зі сніпетами.

9. NavigationService

```
1 package com.example.text_editor.service;
2
3 import com.example.text_editor.model.Bookmark;
4 import com.example.text_editor.repository.BookmarkRepository;
5 import org.springframework.stereotype.Service;
6
7 import java.util.List;
8
9 @Service
10 public class NavigationService {
11
12     3 usages
13     private final BookmarkRepository bookmarkRepository;
14
15     1 usage
16     public NavigationService(BookmarkRepository bookmarkRepository) { this.bookmarkRepository = bookmarkRepository; }
17
18     1 usage
19     public Bookmark addBookmark(String fileName, int line, String note) {
20         return bookmarkRepository.save(new Bookmark(fileName, line, note));
21     }
22
23     1 usage
24     public List<Bookmark> getBookmarks(String fileName) { return bookmarkRepository.findByFileName(fileName); }
25 }
```

Рисунок 12. Клас NavigationService.

Цей клас, показаний на Рисунку 12, відповідає за функціонал навігації. Метод addBookmark дозволяє створювати закладки для файлів із вказаними рядками та нотатками. Метод getBookmarks повертає список закладок для заданого файлу.

10. NavigationController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.model.Bookmark;
4 import com.example.text_editor.service.NavigationService;
5 import org.springframework.web.bind.annotation.*;
6
7 import java.util.List;
8
9 @RestController
10 @RequestMapping("/navigation")
11 public class NavigationController {
12
13     3 usages
14     private final NavigationService navigationService;
15
16     1 usage
17     public NavigationController(NavigationService navigationService) { this.navigationService = navigationService; }
18
19     @PostMapping("/bookmark")
20     public Bookmark addBookmark(@RequestParam String fileName, @RequestParam int line, @RequestParam String note) {
21         return navigationService.addBookmark(fileName, line, note);
22     }
23
24     @GetMapping("/bookmarks")
25     public List<Bookmark> getBookmarks(@RequestParam String fileName) {
26         return navigationService.getBookmarks(fileName);
27     }
28 }
```

Рисунок 13. Клас NavigationController.

Цей контролер, показаний на Рисунку 13, надає API для роботи із закладками. Метод `addBookmark` дозволяє додавати нові закладки для файлів, а метод `getBookmarks` повертає список закладок для заданого файлу.

11. SyntaxHighlightService

```
1 package com.example.text_editor.service;
2
3 import org.springframework.stereotype.Service;
4
5 no usages
6 @Service
7 public class SyntaxHighlightService {
8
9     no usages
10    @
11    public String highlightSyntax(String content) {
12        return content.replaceAll(regex: "public", replacement: "[PUBLIC]")
13        .replaceAll(regex: "class", replacement: "[CLASS]");
14    }
15 }
```

Рисунок 14. Клас `SyntaxHighlightService`.

Цей клас, показаний на Рисунку 14, реалізує підсвічування синтаксису для тексту. Метод `highlightSyntax` підсвічує ключові слова, такі як `public` або `class`, у тексті.

12. SyntaxHighlightController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.strategy.*;
4 import org.springframework.web.bind.annotation.*;
5
6 @RestController
7 @RequestMapping("/syntax")
8 public class SyntaxHighlightController {
9
10     5 usages
11     private final SyntaxHighlightContext context;
12     2 usages
13     private final JavaSyntaxHighlightStrategy javaStrategy;
14     2 usages
15     private final HtmlSyntaxHighlightStrategy htmlStrategy;
16     2 usages
17     private final PlainTextHighlightStrategy plainStrategy;
18
19     public SyntaxHighlightController(SyntaxHighlightContext context,
20                                     JavaSyntaxHighlightStrategy javaStrategy,
21                                     HtmlSyntaxHighlightStrategy htmlStrategy,
22                                     PlainTextHighlightStrategy plainStrategy) {
23         this.context = context;
24         this.javaStrategy = javaStrategy;
25         this.htmlStrategy = htmlStrategy;
26         this.plainStrategy = plainStrategy;
27     }
28 }
```

Рисунок 15.1. Клас SyntaxHighlightController.

```
25 @PostMapping("/highlight")
26 public String highlight(@RequestParam String text, @RequestParam String type) {
27     switch (type.toLowerCase()) {
28         case "java":
29             context.setStrategy(javaStrategy);
30             break;
31         case "html":
32             context.setStrategy(htmlStrategy);
33             break;
34         default:
35             context.setStrategy(plainStrategy);
36     }
37     return context.highlight(text);
38 }
39 }
```

Рисунок 15.2. Клас SyntaxHighlightController.

Цей контролер, показаний на Рисунках 15.1 та 15.2, надає API для підсвічування синтаксису. Метод `highlight` приймає текст і тип (наприклад, `java`, `html`) і викликає відповідну стратегію для підсвічування синтаксису.

13. DefaultController

```
1 package com.example.text_editor.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class DefaultController {
8
9     @GetMapping("/")
10    public String index() {
11        return "Welcome to the Text Editor API! Available endpoints: " +
12            "/file, /editor, /syntax, /navigation";
13    }
14 }
```

Рисунок 16. Клас DefaultController.

Цей клас, показаний на Рисунок 16, є базовим контролером для текстового редактора. Він надає точку входу в систему через ендпойнт /. Метод index повертає привітальне повідомлення, яке містить перелік доступних ендпойнтів API: /file, /editor, /syntax, /navigation.

Посилання на код на GitHub: https://github.com/vvvvvvvvvviktoriiia/text_editor

Діаграма класів архітектури «SOA»:

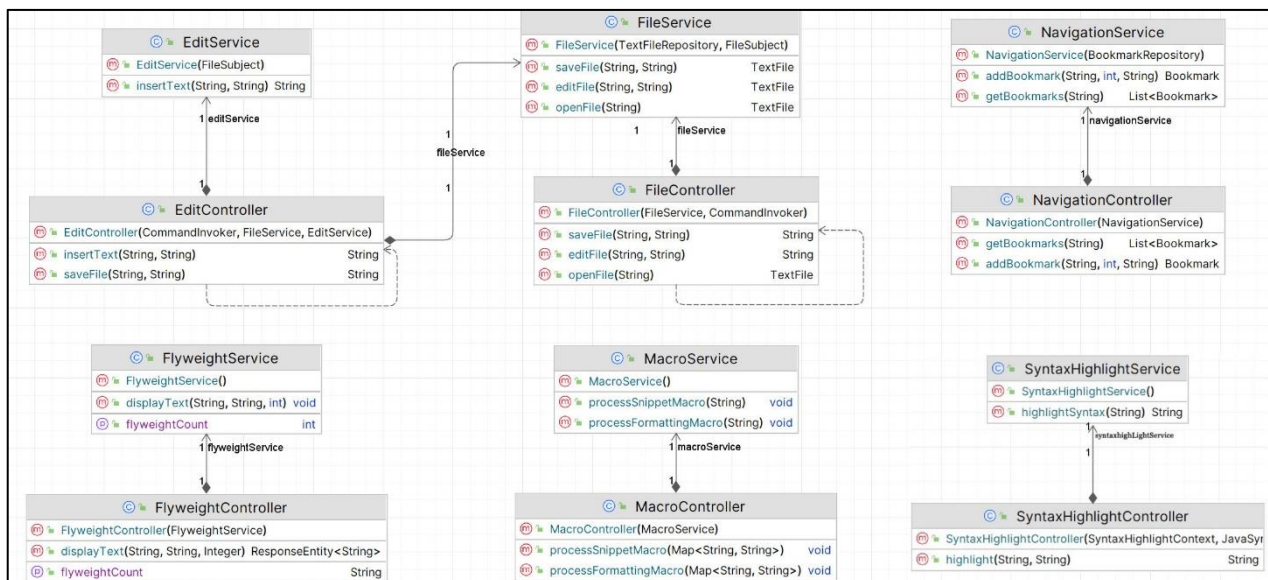


Рисунок 17. Діаграма класів архітектури «SOA».

У текстовому редакторі реалізовано архітектуру Service-Oriented Architecture (SOA), яка дозволила досягти модульності, масштабованості та гнучкості в роботі системи. Основна ідея SOA — розділення функціоналу на незалежні сервіси з чітко визначеними інтерфейсами. Раніше складні, монолітні рішення ускладнювали підтримку та розвиток системи, що призводило до:

- Тісної залежності між компонентами: Зміна одного з компонентів могла спричинити поломку інших.
- Складності в тестуванні: Всі функції працювали у спільному середовищі, що ускладнювало модульні тести.
- Проблем із масштабуванням: Збільшення навантаження вимагало змін у всій системі.

Архітектура SOA вирішила ці проблеми, дозволивши:

- Розділення функціоналу: Кожен сервіс виконує чітко визначену задачу.
- Легке масштабування: Сервіси можуть працювати незалежно один від одного, що дозволяє збільшувати продуктивність окремих компонентів без змін у системі.
- Гнучке розширення: Нові функції додаються через окремі сервіси без порушення роботи інших.

Переваги застосування архітектури «SOA»:

1. Модульність:

- Кожен сервіс ізольований, що спрощує його підтримку, тестування та розширення.

2. Повторне використання:

- Один сервіс може використовуватись у різних частинах системи або навіть у різних проектах.

3. Масштабованість:

- Система легко адаптується до зростання навантаження за рахунок незалежного масштабування сервісів.

4. Незалежність:

- Сервіси взаємодіють через API, що дозволяє їх розробляти, тестувати та оновлювати незалежно.

5. Гнучкість:

- Можливість додавання нових функцій без змін у існуючій логіці.

Висновок: У даній лабораторній роботі реалізовано систему текстового редактора з використанням архітектури SOA. Логіка роботи була розділена на сервіси для управління файлами, редагування тексту, навігації, підсвічування синтаксису, роботи з макросами та шрифтами. Це забезпечило гнучкість, зручність у розвитку та ефективність використання ресурсів.

Архітектура SOA зробила систему легко масштабованою, дозволила забезпечити незалежність компонентів і повторне використання сервісів. Завдяки цьому текстовий редактор став продуктивним, зручним для подальшого розвитку та підтримки.